

1.

- a. The two AI's used in this code are ChatGPT and Gemini.
- b. They were accessed through my computer on the web browser
- c. The prompt used for both AI's: Write the program in C and make sure to add comments on every line explaining what that line code does
- d. Code from ChatGPT:

```
#include <stdio.h> // Includes the standard input/output library so we can use printf() and scanf()
```

```
int main() // Starts the main function where the program begins running
{ // Begins the main function's block
```

```
    int secretNumber = 7; // Stores the fixed secret number that the user needs to guess
```

```
    int guess; // Creates a variable to store the user's guess
```

```
    int attempts = 0; // Keeps track of how many guesses the user has made
```

```
    int correct = 0; // Keeps track of whether the user guessed the number correctly
```

```
    printf("Guess a number between 1 and 10.\n"); // Tells the user what number range they should guess
```

```
    while (attempts < 3 && correct == 0) // Repeats the loop while the user has fewer than 3 attempts and has not won
```

```
    { // Begins the while loop
```

```
        attempts++; // Adds 1 to the number of attempts before asking for a guess
```

```
        printf("Attempt %d/3. Enter your guess: ", attempts); // Tells the user which attempt they are on and asks for their guess
```

```
        scanf("%d", &guess); // Reads the user's guess and stores it in the guess variable
```

```
        if (guess == secretNumber) // Checks if the user's guess is equal to the secret number
```

```
        { // Begins the correct-answer block
```

```
        printf("Correct! You win!\n"); // Tells the user that they guessed the
number correctly
```

```
        correct = 1; // Changes correct to 1 to show that the user won
```

```
    } // Ends the correct-answer block
```

```
    else if (guess < secretNumber) // Checks if the user's guess is lower
than the secret number
```

```
    { // Begins the too-low block
```

```
        printf("Too low! Try again.\n"); // Tells the user that their guess was
too low
```

```
    } // Ends the too-low block
```

```
    else // Runs when the guess is not correct and is not too low
```

```
    { // Begins the too-high block
```

```
        printf("Too high! Try again.\n"); // Tells the user that their guess
was too high
```

```
    } // Ends the too-high block
```

```
    } // Ends the while loop
```

```
    if (correct == 0) // Checks if the user never guessed the secret number
correctly
```

```
    { // Begins the losing block
```

```
        printf("You lose! The secret number was %d.\n", secretNumber); //
Tells the user they lost and reveals the secret number
```

```
    } // Ends the losing block
```

```
    return 0; // Ends the program and tells the computer that the program
ran successfully
```

```
    } // Ends the main function
```

Code from Gemini:

```
#include <stdio.h> // Include the standard input/output library for printf and scanf
```

```
int main() { // Define the main function where execution begins
    const int SECRET_NUMBER = 7; // Store the target secret number to be guessed
```

```
    const int MAX_ATTEMPTS = 3; // Define the maximum number of allowed attempts
```

```
    int guess; // Declare a variable to store the user's current guess
```

```
    int won = 0; // Initialize a flag variable (0 for lost, 1 for won)
```

```
    printf("Guess a number between 1 and 10.\n"); // Print the opening instruction message
```

```
    for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) { // Loop from attempt 1 up to MAX_ATTEMPTS
```

```
        printf("Attempt %d/%d. Enter your guess: ", attempt,
```

```
        MAX_ATTEMPTS); // Display current attempt prompt
```

```
        scanf("%d", &guess); // Read the integer input entered by the user
```

```
        if (guess == SECRET_NUMBER) { // Check if the user guessed the secret number correctly
```

```
            printf("Correct! You win!\n"); // Print winning feedback message
```

```
            won = 1; // Mark the outcome flag as won
```

```
            break; // Immediately exit the loop since the correct answer was found
```

```
        } else if (guess < SECRET_NUMBER) { // Check if the guess was lower than the secret number
```

```
            if (attempt < MAX_ATTEMPTS) { // Check if there are remaining attempts left
```

```
                printf("Too low! Try again.\n"); // Print prompt to try a higher number
```

```
            } else { // Handle the case when no attempts remain
```

```
                printf("Too low!\n"); // Print final feedback message for a low guess
```

```
            } // End of inner low-guess condition check
```

```
        } else { // Handle the case where the guess was higher than the secret number
```

```

        if (attempt < MAX_ATTEMPTS) { // Check if there are remaining
attempts left
            printf("Too high! Try again.\n"); // Print prompt to try a lower
number
        } else { // Handle the case when no attempts remain
            printf("Too high!\n"); // Print final feedback message for a high
guess
        } // End of inner high-guess condition check
    } // End of outer check for incorrect guesses
} // End of the guessing loop

if (!won) { // Check if the player finished all attempts without winning
    printf("You ran out of attempts. You lose!\n"); // Print game over loss
message
} // End of loss condition check

return 0; // Return 0 to indicate that the program executed successfully
} // End of the main function

```

- e. Correctness: Both programs are able to run and finish with no errors in typical use cases. When inputting anything that is not a number both programs immediately use all tries but does not crash. Execution time: Gemini a for-loop for the amount of tires with if-else statements for what to do in each scenario (wrong/right guesses). This will be more efficient then ChatGPT uses a while loop with if-else statements. Space complexity: Gemini used two constants and two integers. This combined with the for-loop will be more efficient then compared to ChatGPT. ChatGPT uses four integers which combined with the use of a while-loop will lead to more memory usage and processing power. Maintainability: All programs have comments that explain what every line of code is doing. A deeper analysis of both programs show that it cannot handle unexpected inputs (letter where number should be).
- f. I'm selecting the Gemini as it beats out ChatGPT in terms of correctness, execution time, space complexity, and maintainability. As well it is much easier to read and decipher than ChatGPT.
- g. Add a verification to the code that checks the input if it is a number and prompts the user to try again if it is not. This helps with maintainability as the program now has a way to handle an incorrect input.